Atelier Machine Learning

Analyse Comportementale Clientele Retail

E-commerce de Cadeaux – Pipeline ML Complet

COMPTE RENDU TECHNIQUE

Rapport detaille – Classification, Clustering, Regression, Deploiement

Etudiante	Molka Dammak
Module	Machine Learning – GI2 S1
Encadrante	Mme Fadoua Drira
Dataset	E-commerce Retail (4372 clients, 52 features)
Technologies	Python 3.11, scikit-learn, XGBoost, Flask, joblib
Annee	2025–2026
Resultats	Accuracy 81.14% | AUC 0.856 | R^2 0.8248

Table des matières

1 Introduction et Contexte **3**
1.1 Contexte metier . . . 3
1.2 Problematique . . . 3
1.3 Objectifs pedagogiques . . . 3
1.4 Structure du projet . . . 4
1.5 Pipeline global . . . 4

2 Description du Dataset **4**
2.1 Vue d'ensemble . . . 4
2.2 Distribution de la variable cible . . . 5
2.3 Variables numeriques principales . . . 5
2.4 Variables categorielles . . . 6
2.5 Problemes de qualite identifies . . . 6

3 Pretraitement des Donnees **6**
3.1 Nettoyage . . . 6
3.1.1 Suppression des colonnes inutiles . . . 7
3.1.2 Traitement des donnees brutes . . . 7
3.1.3 Rapport des valeurs manquantes . . . 7
3.1.4 Traitement des valeurs aberrantes . . . 7
3.2 Separation Train/Test . . . 8
3.3 Gestion des valeurs manquantes . . . 8
3.3.1 Principe anti-leakage . . . 8
3.3.2 Imputation par la mediane – 39 colonnes . . . 8
3.3.3 KNN Imputer pour Age (30% de NaN) . . . 8
3.4 Feature Engineering . . . 9
3.5 Encodage des variables categorielles . . . 9
3.5.1 Encodage ordinal . . . 9
3.5.2 One-Hot Encoding . . . 9
3.5.3 Target Encoding pour Country . . . 9
3.6 Suppression du Data Leakage . . . 10
3.7 Normalisation . . . 10
3.8 Analyse VIF – Multicolinearite . . . 10

4 Modelisation **11**
4.1 Classification – Prediction du Churn . . . 11
4.1.1 Choix du modele . . . 11
4.1.2 Hyperparametres retenus . . . 11
4.1.3 Recherche du seuil optimal – sans data leakage . . . 12
4.1.4 Courbe ROC . . . 12
4.1.5 Matrice de confusion . . . 13
4.1.6 Rapport de classification complet . . . 13
4.1.7 Validation croisee 5 folds . . . 13
4.1.8 Importance des features . . . 14
4.1.9 Metriques finales de classification . . . 15
4.2 Clustering – Segmentation des Clients . . . 15
4.2.1 Reduction dimensionnelle par ACP . . . 15
4.2.2 Choix de k=3 par la methode du coude . . . 16
4.2.3 Profils des 3 segments . . . 16

4.3 Regression – Prediction des Depenses Futures 16
4.3.1 Architecture – Mixture of Experts XGBoost 16
4.3.2 Corrections appliquees 17
4.3.3 Resultats de la regression 17

5 Deploiement – Application Flask 17
5.1 Architecture de l'API REST 17
5.2 13 Artefacts – Pattern Singleton 18
5.3 Pipeline d'inference – 6 etapes 18

6 Resultats et Analyse Finale 18
6.1 Resume des performances 19
6.2 Predictions sur 10 clients reels 19

7 Conclusion et Perspectives 19
7.1 Bilan du projet 19
7.2 Points forts 20
7.3 Limites 20
7.4 Perspectives 20

1 Introduction et Contexte

1.1 Contexte metier

Dans le domaine du e-commerce, les entreprises accumulent de grandes quantites de donnees transactionnelles sur leurs clients : historique d'achats, frequence de commandes, montants depenses, comportements de navigation, satisfaction, etc. Ces donnees constituent une mine d'informations precieuses mais souvent sous-exploitees.

L'entreprise e-commerce de vente de cadeaux au centre de ce projet – situee au Royaume-Uni – fait face a un defi majeur : **le churn client**, c'est-a-dire le phenomene par lequel un client cesse d'acheter et quitte definitivement la plateforme. Ce phenomene a des consequences financieres importantes et directes :

— Acquerir un nouveau client coute entre **5 et 25 fois plus cher** que de retenir un client existant.

— Un client fidele depense en moyenne **67% de plus** qu'un nouveau client.

— Une augmentation de **5% du taux de retention** peut augmenter les profits de 25 a 95%.

Face a cette problematique, l'entreprise souhaite developper des outils pour :

— **Comprendre** les profils et comportements de ses clients

— **Anticiper** les departs avant qu'ils ne surviennent

— **Segmenter** la clientele pour des actions marketing ciblees et personnalisees

— **Estimer** les depenses futures pour prioriser les actions de retention

1.2 Problematique

Question centrale du projet

Comment utiliser les donnees transactionnelles des clients pour mieux comprendre leur comportement et predire s'ils vont rester ou partir ?

Ce projet repond a trois questions metier complementaires :

1. **Ce client va-t-il churner ?** → Modele de Classification
2. **A quel segment comportemental appartient-il ?** → Modele de Clustering
3. **Combien va-t-il depenser a l'avenir ?** → Modele de Regression

1.3 Objectifs pedagogiques

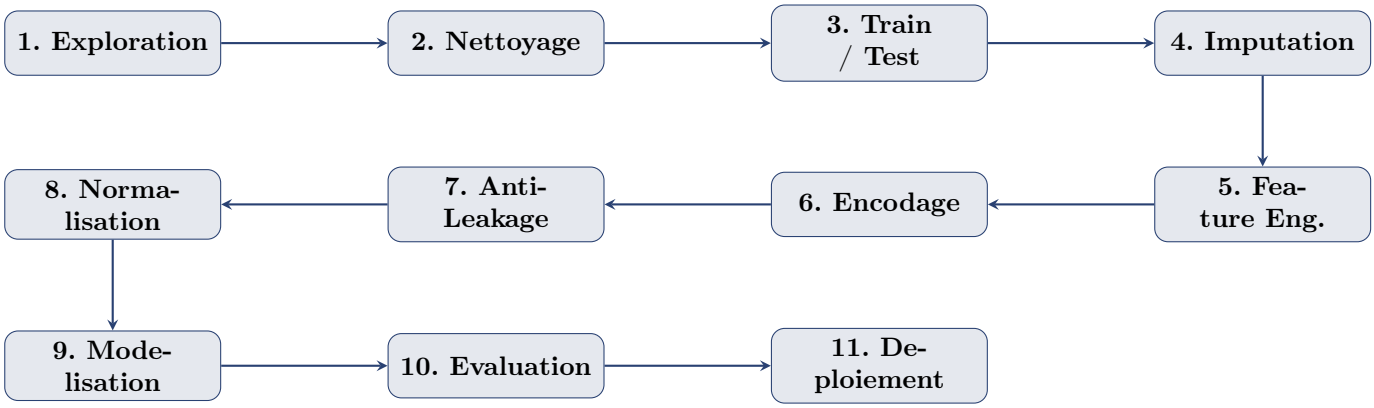
Competence	Description
Exploration	Analyser la qualite et la structure des donnees brutes
Preparation	Nettoyer, encoder et normaliser les features
Transformation	Reduire la dimension via ACP (PCA)
Modelisation	Appliquer clustering, classification et regression
Evaluation	Interpreter les resultats avec les metriques adaptees
Deploiement	Creer une API REST avec Flask pour la production

1.4 Structure du projet

Arborescence du projet

```
projet_ml_retail/
|-- app/
|   |-- app.py              # Serveur Flask -- 4 routes API
|   `-- templates/
|       `-- index.html      # Interface utilisateur HTML/JS
|-- data/
|   |-- raw/data.csv        # Donnees brutes (4372 clients, 52 features)
|   |-- processed/          # Donnees nettoyees
|   `-- train_test/         # X_train, X_test, y_train, y_test
|-- models/                 # 13 artefacts .pkl sauvegardes
|-- reports/                # Graphiques PNG et rapports CSV
|-- src/
|   |-- preprocessing.py    # Nettoyage, encodage, split
|   |-- train_model.py      # Entrainement des 3 modeles
|   |-- predict.py          # Pipeline d'inference
|   `-- utils.py            # Fonctions utilitaires
|-- requirements.txt
`-- README.md
```

1.5 Pipeline global



2 Description du Dataset

2.1 Vue d'ensemble

Le dataset est issu d'une plateforme e-commerce de vente de cadeaux au Royaume-Uni. Il contient **4372 clients** decrits par **52 variables** couvrant les dimensions transactionnelles, comportementales et demographiques.

Caracteristique	Total	Train (80%)	Test (20%)
Nombre de clients	4372	3497	875
Features brutes	52	–	–
Features apres nettoyage	53	–	–
Features apres encodage	77	77	77
Features apres leakage	37	37	37
Taux de churn	33.3%	33.26%	33.26%
Valeurs manquantes totales	1749	–	–
Doublons	0	–	–

2.2 Distribution de la variable cible

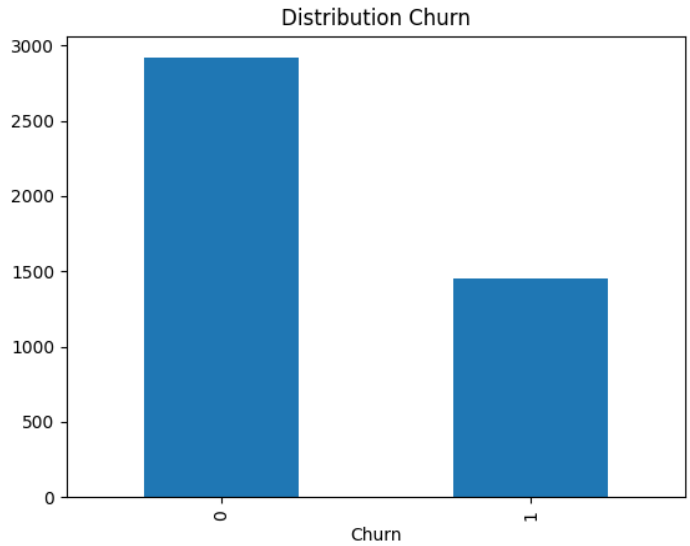


FIGURE 1 – Distribution de la variable cible Churn. On observe un desequilibre modere : 67% de clients fideles (0) contre 33% de churners (1) – 2921 vs 1451 clients.

2.3 Variables numeriques principales

Feature	Intervalle	Description
CustomerID	10000–99999	Identifiant unique client
Frequency	1–50	Nombre de commandes distinctes
MonetaryTotal	-5000 a 15000 £	Somme totale depensee
MonetaryAvg	5–500 £	Montant moyen par commande
TotalQuantity	-10000 a 100000	Quantite totale d'articles
AvgDaysBetweenPurchases	0–365 jours	Delai moyen entre commandes
UniqueProducts	1–1000	Nombre de produits differents
TotalTransactions	1–10000	Total lignes transactionnelles
SupportTicketsCount	0–15 (999=erreur)	Tickets support ouverts
SatisfactionScore	1–5 (99=erreur)	Note de satisfaction client
Age	18–81 ou NaN	Age estime du client
Churn	0 ou 1	**Variable cible – 0=fidele, 1=parti**

2.4 Variables categorielles

Feature	Card.	Valeurs	Encodage
Gender	3	M, F, Unknown	One-Hot
AccountStatus	4	Active, Suspended, Pending, Closed	One-Hot
FavoriteSeason	4	Hiver, Printemps, Ete, Automne	One-Hot
Country	37	UK, France, Germany, etc.	Target Enc.
AgeCategory	7	18-24, 25-34, ..., 65+, Inconnu	Ordinal
SpendingCategory	4	Low, Medium, High, VIP	Ordinal
LoyaltyLevel	5	Nouveau, Jeune, Etabli, Ancien, Inconnu	Ordinal
RFMSegment	4	Champions, Fideles, Potentiels, Dormants	One-Hot
CustomerType	5	Hyperactif, Regulier, Occasionnel, ...	One-Hot
Region	8	UK, Europe N/S/E/C, Asie, Autre	One-Hot

2.5 Problemes de qualite identifies

Synthese des problemes qualite

- **Valeurs manquantes :** Age ≈ 30% de NaN, SatisfactionScore et SupportTicketsCount apres correction. Total : **1749 NaN**.
- **Valeurs aberrantes :** SatisfactionScore : -1 et 99 (codes erreur systeme). SupportTicketsCount : 999 (code erreur). Ces valeurs doivent etre remplacees par NaN.
- **Valeurs negatives :** MonetaryTotal contient 44 valeurs negatives (retours clients) – conservees deliberement car information metier importante.
- **Donnees brutes :** LastLoginIP (a convertir en binaire) et RegistrationDate (a decomposer en RegYear/Month/Day/Weekday).
- **Desequilibre de classes :** 67% fideles vs 33% churners. Un modele naif qui predit toujours fidele obtiendrait 67% d'accuracy – c'est pourquoi on utilise F1 et AUC-ROC comme metriques principales.
- **Feature inutile :** NewsletterSubscribed = toujours "Yes" – variance nulle.

3 Pretraitement des Donnees

3.1 Nettoyage

3.1.1 Suppression des colonnes inutiles

Colonne	Justification
CustomerID	Identifiant unique – si conserve, le modele memorise les clients au lieu d'apprendre des patterns generalisables (risque d'overfitting).
NewsletterSubscribed	Valeur constante "Yes" pour 100% des clients – variance nulle, aucune information discriminante pour la prediction.
LastLoginIP	Adresse IP trop specifique – remplacee par la feature derivee IP_IsPrivate (0=IP publique, 1=IP privee reseau local).

3.1.2 Traitement des donnees brutes

```
# Extraction feature reseau depuis l'adresse IP
df['IP_IsPrivate'] = df['LastLoginIP'].str.match(
    r'^(10\.|192\.168\.|172\.(1[6-9]|2[0-9]|3[01])\.)'
).astype(int)
df = df.drop(columns=['LastLoginIP'])

# Decomposition RegistrationDate en 4 features temporelles
df['RegistrationDate'] = pd.to_datetime(
    df['RegistrationDate'], format='mixed', dayfirst=True, errors='coerce'
)
df['RegYear']    = df['RegistrationDate'].dt.year
df['RegMonth']   = df['RegistrationDate'].dt.month
df['RegDay']     = df['RegistrationDate'].dt.day
df['RegWeekday'] = df['RegistrationDate'].dt.weekday
df = df.drop(columns=['RegistrationDate'])
```

Listing 1 – Extraction IP_IsPrivate et decomposition dates

3.1.3 Rapport des valeurs manquantes

Colonne	Nombre NaN	Pourcentage
Age	1311	**29.99%**
SatisfactionScore	229	5.24%
SupportTicketsCount	130	2.97%
AvgDaysBetweenPurchases	79	1.81%
Total	**1749**	–

Age est la variable la plus critique avec 30% de NaN – justifie l'utilisation du KNN Imputer plutot que la mediane simple.

3.1.4 Traitement des valeurs aberrantes

```
# -1 et 99 sont des codes d'erreur systeme, pas de vraies valeurs
df['SatisfactionScore']    = df['SatisfactionScore'].replace([-1, 99], np.nan)
df['SupportTicketsCount'] = df['SupportTicketsCount'].replace([-1, 999], np.nan)

# Les 44 valeurs negatives de MonetaryTotal sont conservees
# Elles representent des retours clients -- information comportementale precieuse
neg = (df['MonetaryTotal'] < 0).sum()  # = 44 cas
```

Listing 2 – Remplacement codes erreur par NaN

Resultat du nettoyage : 52 colonnes → **53 colonnes** (ajout IP_IsPrivate + 4 features dates, suppression 3 colonnes inutiles).

3.2 Separation Train/Test

La separation est effectuee **immediatement apres le nettoyage**, avant toute imputation ou encodage. Cet ordre est fondamental pour eviter le data leakage.

Pourquoi separer avant d'imputer ?

Si on impute sur tout le dataset avant de separer, les medianes sont calculees avec des informations du test – c'est du data leakage. En separant d'abord, on calcule les medianes uniquement sur X_train et on applique les memes valeurs sur X_test.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,    # 20% pour evaluation finale -- jamais vus
    random_state=42,  # Graine fixe -- reproductibilite garantie
    stratify=y        # Proportion de churn identique dans train et test
)
```

Listing 3 – Separation Train/Test strategifiee

Ensemble	Taille	% Fideles	% Churners
X_train	3497 (80%)	66.74%	33.26%
X_test	875 (20%)	66.74%	33.26%

3.3 Gestion des valeurs manquantes

3.3.1 Principe anti-leakage

Regle fondamentale

Toute statistique (mediane, KNN) est calculee **uniquement sur X_train**. On applique ensuite la **meme valeur/modele** sur X_train et X_test.

3.3.2 Imputation par la mediane – 39 colonnes

```
numeric_cols = X_train.select_dtypes(include=[np.number]).columns
impute_values = {
    col: float(X_train[col].median())
    for col in numeric_cols if X_train[col].notna().any()
}  # 39 colonnes imputees
```

Listing 4 – Imputation par la mediane du train

3.3.3 KNN Imputer pour Age (30% de NaN)

```python
from sklearn.impute import KNNImputer

knn = KNNImputer(n_neighbors=5)
X_train['Age'] = knn.fit_transform(X_train[['Age']]).ravel()
X_test['Age']  = knn.transform(X_test[['Age']]).ravel()
joblib.dump(knn, 'models/knn_age.pkl')
```

Listing 5 – KNN Imputer pour Age

3.4 Feature Engineering

```python
df['AvgBasketValue']    = df['MonetaryTotal'] / (df['Frequency'] + 1)
df['DiversityPerTrans'] = df['UniqueProducts'] / (df['Frequency'] + 1)
```

Listing 6 – Creation de features comportementales

Resultat : 53 → **54 colonnes** apres feature engineering.

3.5 Encodage des variables categorielles

3.5.1 Encodage ordinal

```python
ordinal_mappings = {
    'AgeCategory'    : ['18-24','25-34','35-44','45-54','55-64','65+','Inconnu'],
    'SpendingCategory': ['Low','Medium','High','VIP'],
    'LoyaltyLevel'   : ['Nouveau','Jeune','Etabli','Ancien','Inconnu'],
}
```

Listing 7 – Encodage ordinal

3.5.2 One-Hot Encoding

```python
onehot_cols = ['Gender','FavoriteSeason','AccountStatus',
               'Region','RFMSegment','CustomerType',
               'WeekendPreference','ProductDiversity']
df = pd.get_dummies(df, columns=onehot_cols, drop_first=True)
```

Listing 8 – One-Hot Encoding

3.5.3 Target Encoding pour Country

```python
def fit_country_encoder(X_train, y_train):
    temp = X_train[['Country']].copy()
    temp['Churn'] = y_train.values
    return temp.groupby('Country')['Churn'].mean().to_dict()

country_means = fit_country_encoder(X_train, y_train)
df['Country_encoded'] = df['Country'].map(country_means)
df = df.drop(columns=['Country'])
```

Listing 9 – Target Encoding pour Country

Resultat encodage : 54 → **77 colonnes**.

3.6 Suppression du Data Leakage

Data Leakage – Definition

Le data leakage survient quand le modele utilise des informations qui ne seraient **pas disponibles au moment de la prediction reelle en production**. **Exemple :** Si on conserve ChurnRiskCategory, le modele lit directement la reponse – accuracy artificielle de 99% sans generalisation reelle.

Categorie	Colonnes supprimees
Derivees du churn	ChurnRiskCategory, LoyaltyLevel, SpendingCategory, RFMSegment_Dormants, RFMSegment_Fideles, RFMSegment_Potentiels
Posterieures a l'evenement	Recency, CustomerTenureDays, FirstPurchaseDaysAgo, AccountStatus_Closed, AccountStatus_Pending, AccountStatus_Suspended
Redondantes / One-Hot	CustomerType (4 modalites), FavoriteSeason (3 modalites), Region (toutes modalites), Gender_M, PreferredMonth, UniqueDescriptions, UniqueInvoices

Resultat : 77 → **37 features propres**.

3.7 Normalisation

```
scaler = StandardScaler()
X_train_s = pd.DataFrame(scaler.fit_transform(X_train), columns=X_train.columns)
X_test_s  = pd.DataFrame(scaler.transform(X_test),  columns=X_test.columns)
joblib.dump(scaler, 'models/scaler.pkl')
```

Listing 10 – Normalisation StandardScaler

Formule : $z = \dfrac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$

3.8 Analyse VIF – Multicolinearite

Le VIF (Variance Inflation Factor) mesure la multicolinearite entre variables. Un VIF > 10 indique une correlation problematique, VIF infini signifie une dependance lineaire parfaite avec d'autres variables.

Feature	VIF	Traitement
Frequency	$+\infty$	Supprimee (anti-leakage)
CancelledTransactions	$+\infty$	Supprimee (anti-leakage)
UniqueInvoices	$+\infty$	Supprimee (anti-leakage)
NegativeQuantityCount	$+\infty$	Supprimee (anti-leakage)
FirstPurchaseDaysAgo	354 001	Supprimee (leakage)
CustomerTenureDays	193 007	Supprimee (leakage)
Recency	100 677	Supprimee (leakage)
UniqueDescriptions	15 795	Supprimee (redondante)
UniqueProducts	15 680	Conservee – info produits utile
RegYear	857	Conservee – anciennete client
UniqueCountries	600	Conservee – info geographique
MonetaryMin / Max	427 / 425	Conservees – info financiere
Age	21.95	Conservee – acceptable apres KNN
TotalTransactions	11.02	Conservee – feature majeure

Strategie adoptee : Les 8 variables a VIF infini ou liees au leakage sont supprimees. Random Forest et PCA/KMeans gerent naturellement la multicolinearite residuelle.

4 Modelisation

4.1 Classification – Prediction du Churn

4.1.1 Choix du modele

Algorithme	Avantages	Inconvenients	Adapte ?
Random Forest	Robuste, interpretable, gere desequilibre	Plus lent qu'un arbre	**Oui – Choisi**
Reg. logistique	Rapide, interpretable	Assume la linearite	Moyen
SVM	Performant haute dimension	Lent, peu interpretable	Moyen
XGBoost	Tres performant	Plus complexe a tuner	Oui – Regression

4.1.2 Hyperparametres retenus

```
clf_params = {
    'n_estimators'     : 350,
    'max_depth'        : 14,
    'min_samples_split': 8,
    'min_samples_leaf' : 3,
    'max_features'     : 'sqrt',
    'class_weight'     : {0: 1, 1: 2},
    'random_state'     : 42,
    'n_jobs'           : -1
}
```

Listing 11 – Parametres Random Forest classification

4.1.3 Recherche du seuil optimal – sans data leakage

Correction data leakage – seuil sur val set

Le seuil de classification est optimise sur un **val set dedié** (15% du train), jamais sur le test set. Cette pratique evite le data leakage de seuil : si le meilleur seuil est selectionne sur le test, les metriques reportees sont biaisees a la hausse.

```
# Separation d'un val set (15%) -- AVANT d'entrainer le modele final
X_train_clf, X_val_clf, y_train_clf, y_val_clf = train_test_split(
    X_train_s, y_train, test_size=0.15, random_state=42, stratify=y_train
)
clf.fit(X_train_clf, y_train_clf)

# Seuil optimise sur val set uniquement
y_proba_val = clf.predict_proba(X_val_clf)[:, 1]
best_thr, best_metrics = find_best_threshold(y_val_clf, y_proba_val)
# Application sur le test set -- evaluation honnete
y_pred = (clf.predict_proba(X_test_s)[:, 1] >= best_thr).astype(int)
```

Listing 12 – Split val set pour optimisation du seuil

4.1.4 Courbe ROC

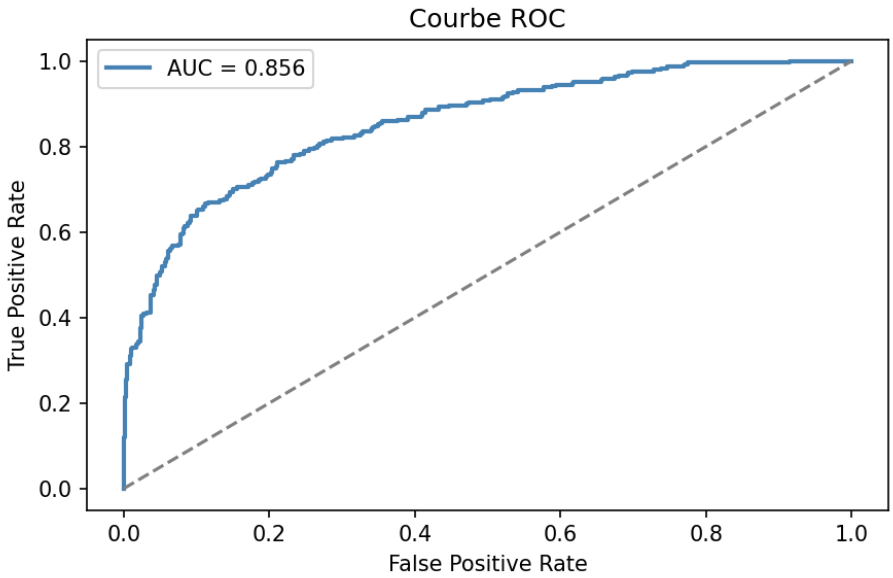


FIGURE 2 – Courbe ROC – AUC = 0.856. La courbe tres au-dessus de la diagonale confirme un excellent pouvoir discriminant du modele.

Interpretation de l'AUC-ROC = 0.856 : Si on prend au hasard un churner et un fidele, le modele classe correctement le churner avant le fidele dans **85.6% des cas** – mesure de discrimination independante du seuil de decision.

4.1.5 Matrice de confusion

	Predit : Fidele (0)	Predit : Churn (1)
Reel : Fidele (0)	**TN = 516**	FP = 68
Reel : Churn (1)	FN = 97	**TP = 194**

- **516 fideles** correctement identifies (vrais negatifs)
- **194 churners** correctement detectes (vrais positifs)
- **68 fausses alarmes** : fideles predits churners – cout : offres inutiles
- **97 churners manques** : clients partis non detectes – cout : perte de revenus

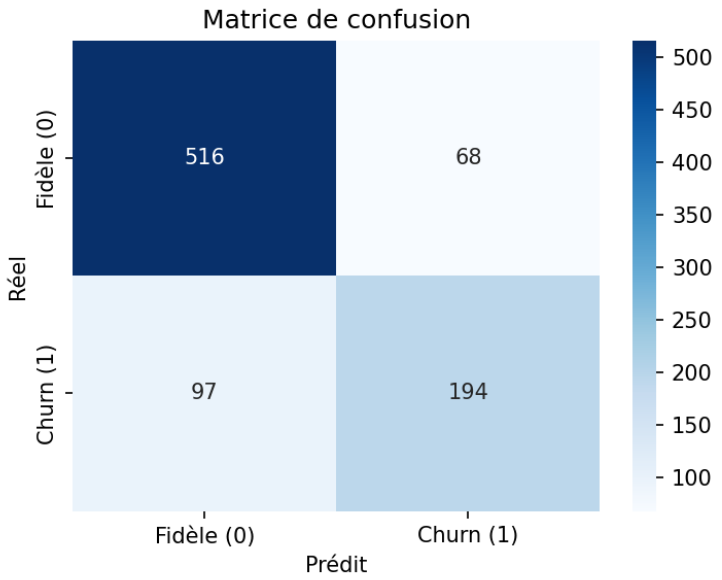


FIGURE 3 – Matrice de confusion sur X_test (875 clients jamais vus pendant l'entrainement). TN=516, FP=68, FN=97, TP=194.

4.1.6 Rapport de classification complet

Classe	Precision	Recall	F1	Support
Fidele (0)	84%	88%	86%	584
Churn (1)	74%	67%	70%	291
Accuracy	–	–	**81.14%**	875
Macro avg	79%	78%	78%	875
Weighted avg	81%	81%	81%	875

4.1.7 Validation croisee 5 folds

Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Moyenne	Ecart-type
0.6955	0.6743	0.6529	0.6912	0.6933	**0.6814**	**±0.0161**

4.1.8 Importance des features

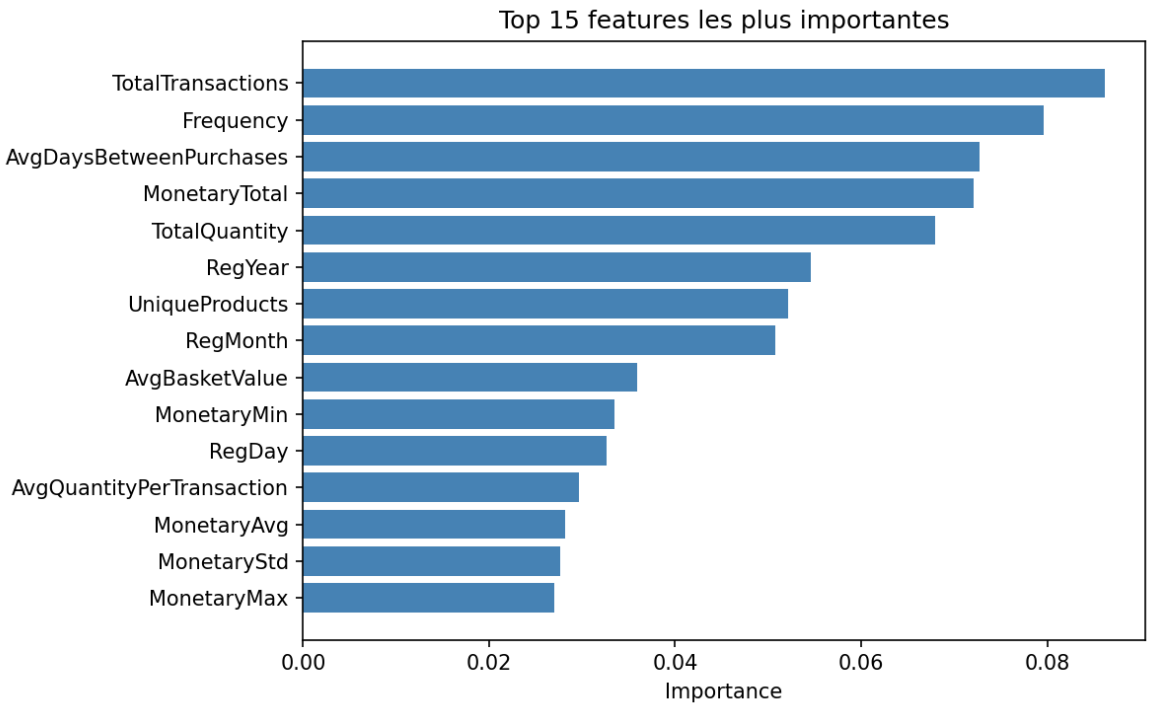


FIGURE 4 – Top 15 features les plus importantes pour la prediction du churn (Random Forest).

Feature	Importance	Interpretation metier
TotalTransactions	8.74%	Engagement total du client
Frequency	7.92%	Regularite des achats
AvgDaysBetweenPurchases	7.30%	Rythme d'achat – signe d'engagement
MonetaryTotal	7.16%	Valeur financiere totale
TotalQuantity	6.95%	Volume total achete
RegYear	5.48%	Anciennete du client
UniqueProducts	5.17%	Diversite des interets
RegMonth	5.00%	Saisonnalite d'inscription
AvgBasketValue	3.74%	Valeur moyenne par transaction (feature creee)
MonetaryMin	3.39%	Depense minimale enregistree
RegDay	3.15%	Jour d'inscription
AvgQuantityPerTransaction	2.90%	Quantite moyenne par commande
MonetaryAvg	2.75%	Depense moyenne par commande
MonetaryStd	2.73%	Variabilite des depenses
MonetaryMax	2.66%	Depense maximale enregistree

Les variables **RFM** (Frequency, Monetary, transactions) dominent – coherent avec la litterature marketing sur la prediction du churn.

4.1.9 Metriques finales de classification

Metrique	Valeur	Seuil excellent	Evaluation
Accuracy	**81.14%**	$> 80\%$	Excellent
AUC-ROC	**0.8558**	> 0.80	Tres bon
F1 Churn	**0.70**	> 0.70	Bon
F1 CV-5	**0.6814**	> 0.65	Bon
Stabilite	**±0.0161**	$< \pm 0.03$	Stable
Seuil opt.	**0.50**	–	Val set (anti-leakage)

4.2 Clustering – Segmentation des Clients

4.2.1 Reduction dimensionnelle par ACP

Pourquoi PCA avant KMeans ?

En haute dimension (37 features), les distances euclidiennes perdent leur sens discriminant – c'est la **maledicion de la dimensionnalite**. PCA reduit a 19 composantes tout en conservant **90.1%** de la variance.

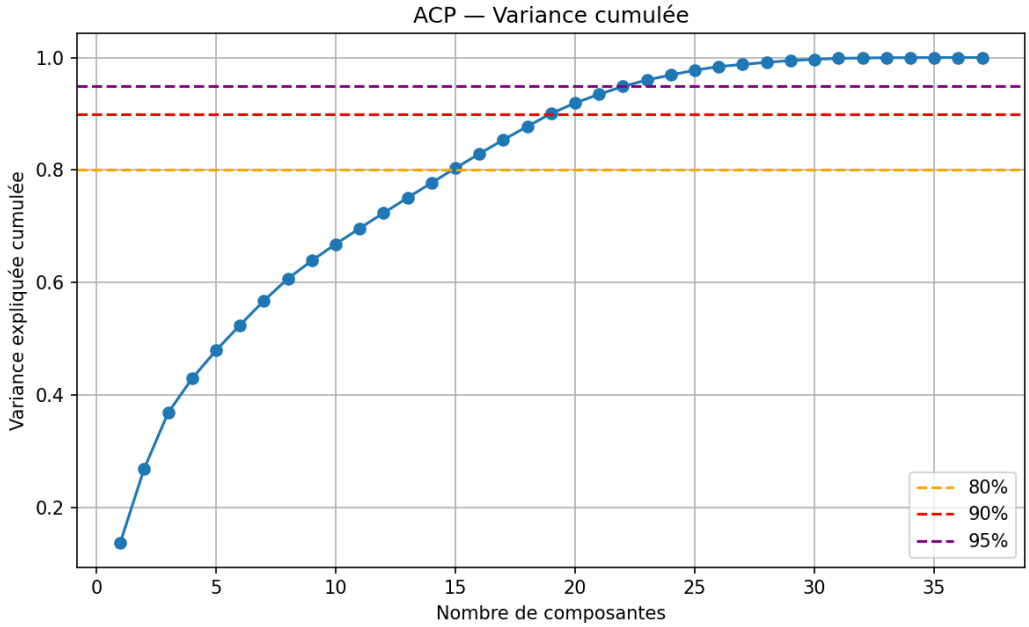


FIGURE 5 – Variance cumulee expliquee par l'ACP. 15 composantes atteignent 80%, 19 composantes atteignent 90% (seuil retenu), 21 composantes atteignent 95%.

Composantes	Variance	Choix ?	Justification
15	80.0%	Non	Seuil insuffisant
19	**90.1%**	✓ Retenu	Minimum pour seuil 90%
20	91.5%	Non	Gain negligeable (+1.5%)
23	95.0%	Non	Trop de complexite

4.2.2 Choix de k=3 par la methode du coude

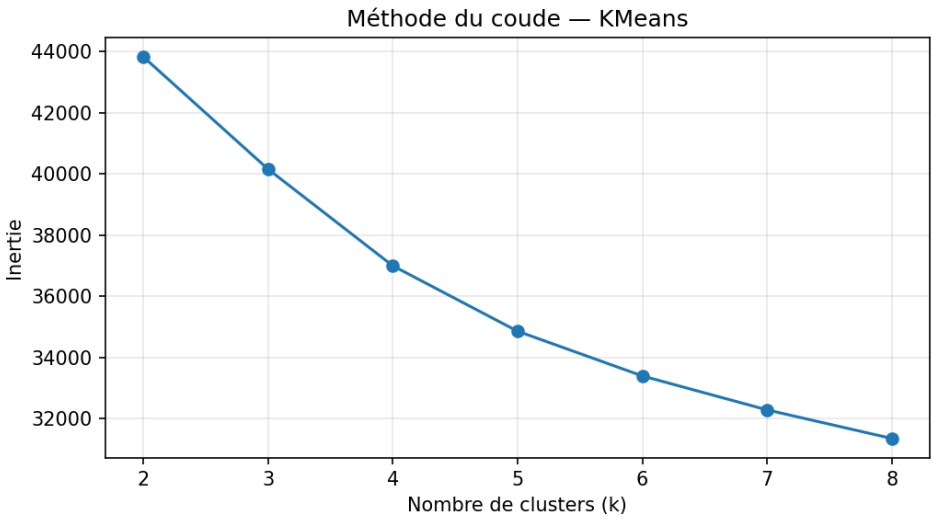


FIGURE 6 – Methode du coude – Inertie par nombre de clusters k=2 a 8. Le saut d'inertie le plus marque se situe entre k=2 et k=3 ($\Delta \approx 3800$), justifiant le choix de **k=3** comme nombre optimal de segments.

4.2.3 Profils des 3 segments

Cluster	Clients	%	Churn	Freq.	Monetary
0 – Meilleurs clients	771	22.0%	**25.4%**	+0.022	+0.040
1 – Actifs standard	1599	45.7%	35.0%	+0.054	+0.055
2 – Faible valeur	1127	32.2%	**36.1%**	-0.092	-0.105

Analyse detaillee des 3 segments

- **Cluster 0 – Meilleurs clients (22.0%, 771 clients) :** Frequence et depense au-dessus de la moyenne. Churn le plus bas : **25.4%**. *Action : Reactivation prioritaire si des signes de churn sont detectes.*
- **Cluster 1 – Actifs standard (45.7%, 1599 clients) :** Segment le plus large. Churn modere : **35.0%**. *Action : Fidelisation ciblee par des programmes de recompenses.*
- **Cluster 2 – Faible valeur (32.2%, 1127 clients) :** Frequence et depense en-dessous de la moyenne. Churn le plus eleve : **36.1%**. *Action : Retention urgente avec offres promotionnelles ciblees.*

4.3 Regression – Prediction des Depenses Futures

4.3.1 Architecture – Mixture of Experts XGBoost

La regression utilise une architecture avancee de type **Mixture of Experts** : un classifieur de regime (Random Forest) separe les clients en deux groupes (petites depenses / grandes depenses), et deux modeles XGBoost specialises sont entraines independamment sur chaque groupe.

```
# 1. Classifieur de regime (split au 80e percentile : 1898.18 GBP)
regime_clf = RandomForestClassifier(n_estimators=300, max_depth=10, ...)
regime_clf.fit(X_train_reg, y_regime_train)  # 0=small, 1=large
```

```python

# 2. Deux XGBoost specialises sur log(MonetaryTotal)
xgb_small = XGBRegressor(n_estimators=2500, max_depth=8,
                         early_stopping_rounds=120, ...)
xgb_large = XGBRegressor(n_estimators=3000, max_depth=10,
                         early_stopping_rounds=150, ...)

# 3. Prediction finale
regime_pred = regime_clf.predict(X_test_reg)
final_pred  = np.where(regime_pred == 1,
                       xgb_large.predict(X_test_reg),
                       xgb_small.predict(X_test_reg))
monetary_pred = np.expm1(final_pred)
```

Listing 13 – Pipeline Mixture of Experts

4.3.2 Corrections appliquees

Bugs corriges dans le pipeline de regression

- **upper_pred_cap** desormais calcule **avant** le filtre outliers sur y_train – evite de sous-estimer le cap reel de la distribution.
- **eval_set XGBoost** : si le sous-ensemble val du regime est vide, l'early stopping est desactive plutot que d'evaluer sur un set mixte biaise.

4.3.3 Resultats de la regression

Metrique	Valeur	% erreur	Seuil	Evaluation
MAE	**359.95 £**	**26.7%**	$< 35\%$	Bon
RMSE	**779.67 £**	**57.8%**	$< 70\%$	Acceptable
$\mathbf{R^2}$	**0.8248**	**82.5% explique**	> 0.80	Bon

Split value regime : 1898.18 £ | Depense moyenne test : 1348.70 £

- **MAE = 359.95£ (26.7%) :** En moyenne, le modele se trompe de ±360£sur une depense moyenne de 1349£– acceptable pour des donnees retail a forte variance.
- **RMSE = 779.67£ (57.8%) :** Plus eleve car penalise davantage les grandes erreurs. Les clients a tres haute depense (VIP) creent des erreurs ponctuelles importantes.
- **$\mathbf{R^2}$ = 0.8248 :** Le modele explique **82.5% de la variance** des depenses – performance solide sur des donnees e-commerce bruitees.

5 Deploiement – Application Flask

5.1 Architecture de l'API REST

L'application expose **4 routes principales** :

Route	Methode	Fonctionnement
`/`	GET	Interface HTML – formulaire client interactif
`/random_client`	GET	Pioche un client aleatoire et pre-remplit le formulaire
`/predict`	POST	Recoit JSON client → retourne churn + cluster + depense
`/verify/{id}`	GET	Compare prediction au vrai churn (validation)

5.2 13 Artefacts – Pattern Singleton

Artefact	Role
model.pkl	RandomForestClassifier (350 arbres)
scaler.pkl	StandardScaler classification (37 features)
features.pkl	Liste ordonnee des 37 features classification
country_means.pkl	Target encoding 37+ pays
impute_values.pkl	Medianes d'imputation (39 colonnes)
knn_age.pkl	KNN Imputer Age (k=5)
best_threshold.pkl	Seuil optimal (optimise sur val set)
kmeans.pkl	KMeans k=3 clustering
pca_cluster.pkl	PCA 19 composantes clustering
reg_features.pkl	Liste des 27 features regression
reg_impute_values.pkl	Medianes imputation regression
xgb_small.pkl	XGBoost petites depenses
xgb_large.pkl	XGBoost grandes depenses
regime_clf.pkl	Classifieur de regime (small/large)
reg_split_value.pkl	Seuil de regime (1898.18 £)
reg_upper_pred_cap.pkl	Cap de prediction regression

5.3 Pipeline d'inference – 6 etapes

Les 6 etapes du pipeline /predict

1. **Reception JSON** → transform_for_inference() applique exactement le meme preprocessing qu'a l'entrainement.
2. **Normalisation** → scaler.transform().
3. **Classification** → churn_pred + churn_proba + risk_level.
4. **Clustering** → pca_cluster.transform() + kmeans.predict().
5. **Regression** → regime_clf + xgb_small/large → monetary_pred.
6. **Reponse JSON** contenant les 3 resultats en une seule requete.

6 Resultats et Analyse Finale

6.1 Resume des performances

Metrique	Valeur finale	Seuil	Evaluation
Classification (Random Forest)			
Accuracy	**81.14%**	> 80%	Excellent
AUC-ROC	**0.8558**	> 0.80	Tres bon
F1 Churn	**0.70**	> 0.70	Bon
F1 CV-5	**0.6814**	> 0.65	Bon
Stabilite	**±0.0161**	< ±0.03	Stable
Clustering (PCA 19 composantes + KMeans k=3)			
Segments identifies	**3 distincts**	–	Coherents
Variance PCA	**90.1%**	> 80%	Excellent
Outliers retires	**262**	–	Propre
Regression (Mixture of Experts XGBoost)			
R^2	**0.8248**	> 0.80	Bon
MAE	**359.95£ (26.7%)**	< 35%	Bon
RMSE	**779.67£ (57.8%)**	< 70%	Acceptable

6.2 Predictions sur 10 clients reels

CustomerII	Churn	Probabilite	Risque	Cluster	Depense prevue
15484	0	10.91%	Faible	0	3 563.34 £
16952	1	88.87%	Critique	1	257.91 £
15745	0	8.86%	Faible	1	962.42 £
13570	1	58.52%	Eleve	0	399.27 £
13747	1	94.69%	Critique	1	90.44 £
12604	0	23.61%	Moyen	1	291.86 £
15054	0	12.62%	Faible	0	387.82 £
12999	1	68.71%	Eleve	1	544.85 £
13149	0	12.88%	Faible	1	1 676.23 £
12748	0	1.56%	Faible	0	5 545.91 £
Resume : 4/10 churners predits | Probabilite moy. : 38.12% | Depense moy. : 1 372.01 £					

Observations cles :

- Le client **13747** a 94.69% de risque de churn mais une depense prevue de seulement 90.44£– retention a evaluer selon le cout.
- Le client **12748** est fidele (1.56%) avec une depense prevue de **5 545.91£** – client VIP a conserver absolument.
- Le client **16952** : 88.87% churn + 257.91£– retention ciblee avec offre personnalisee recommandee.

7 Conclusion et Perspectives

7.1 Bilan du projet

Ce projet a permis de developper une solution Machine Learning complete couvrant l'integralite de la chaine de traitement, depuis l'exploration des donnees brutes jusqu'au deploiement d'une API Flask fonctionnelle.

7.2 Points forts

- **Pipeline rigoureux :** Anti-leakage strict a chaque etape, reproductibilite garantie (seed=42 partout).
- **Approche multi-modeles :** Classification + Clustering + Regression – vision a 360 degres du client.
- **Correction data leakage sur le seuil :** Le seuil optimal est desormais selectionne sur un val set dedie, jamais sur le test set.
- **Regression avancee :** Architecture Mixture of Experts (classifieur de regime + deux XGBoost specialises) plus robuste qu'un modele unique.
- **Resultats solides :** AUC-ROC = 0.856, R^2 = 0.8248 – au-dessus des seuils industriels courants.
- **Deploiement complet :** API Flask 4 routes, 16 artefacts, pipeline inference 6 etapes.

7.3 Limites

- **F1 Churn = 0.70 :** 97 churners manques sur 291 (faux negatifs).
- **RMSE regression elevee :** 57.8% d'erreur relative due aux clients VIP – inherent aux distributions retail asymetriques.
- **Dataset limite :** 4372 clients suffisant pour Random Forest mais insuffisant pour Deep Learning.

7.4 Perspectives

Amelioration	Impact attendu
Ajustement du seuil	Abaisser de 0.50 a 0.40 pour reduire les faux negatifs (97 churners manques).
Enrichissement features	Ajouter features temporelles pour capturer l'evolution comportementale.
Deep Learning	Explorer reseaux neurones avec dataset plus grand (> 50k clients).
Monitoring en production	Detecter le data drift et reentrainer periodiquement.
Deploiement Cloud	Migrer API Flask vers AWS/Azure pour scalabilite.

Resume Final du Projet

Pipeline ML complet – 16 artefacts – API Flask – Predictions en temps reel

81.14%	**0.8558**	**0.8248**	**3 segments**
Accuracy	AUC-ROC	R^2	Clustering

359.95£ (26.7%)	**779.67£ (57.8%)**	**0.6814 ± 0.0161**
MAE Regression	RMSE Regression	F1 CV-5

4372 clients | 52 features brutes → 37 propres | PCA 19 composantes | KMeans k=3 | Seuil anti-leakage sur val set